1.SCENARIO 1: *SEARCH FUNCTIONALITY*

Scenario Name

SEARCH FUNCTIONALITY //Write the scenario you have chosen here

Weak Prompt

"Test the search feature." //Copy the weak prompt from the scenario card above

What I Feel is Missing

NOT MENTIONED WHICH TYPE OF APPLICATION,WHICH VALUE TO SEARCH,WHICH TOOL,WHICH LANGUAGE //List the gaps you identified in the weak prompt

Instruction

//What exactly do you want the AI to do?

Context

//What is the application, environment, or domain?

Example

//Give one sample test case or scenario as a reference

Persona

//Who is the AI acting as? (e.g., Senior QA Engineer)

Output Format

//How should the output be structured? (Table, Gherkin, numbered list)

Tone

//Formal, Technical, Simple — who is the audience?

✅ **Rewritten Prompt**

//Combine all ICEPOT components into your final prompt here

Instruction
1.Create automation script for search field.
2. Create positive and negative testcase to validate the result.
3. Use playwright tool with typescript as language. Use test tags to execute positive scenario.
4. Make sure to add comments.
5. Make sure to follow the coding best practices.
Context
Check search functionality amazon application
Example

```
import test from "playwright/test";
test("Login page ",async({page})=>{
   await page.goto("http://leaftaps.com/opentaps/control/main")
```

```
  await page.locator("#username").fill("DemoSalesManager")
  await page.fill("#password","crmsfa")
 // await page.locator("#password").fill("crmsfa")
  await page.click(".decorativeSubmit")
  // await page.locator(".decorativeSubmit").click
  await page.click(`text='CRM/SFA'`)
  //get title of the page
  const pageTitle=await page.title()
  console.log(pageTitle)

  await page.waitForTimeout(5000)

})
```

Persona

As a Senior automation test engineer

Output Format

Logical code should be in pages.

Test class should use the methods in pages class.

Page Object model format.

Tone

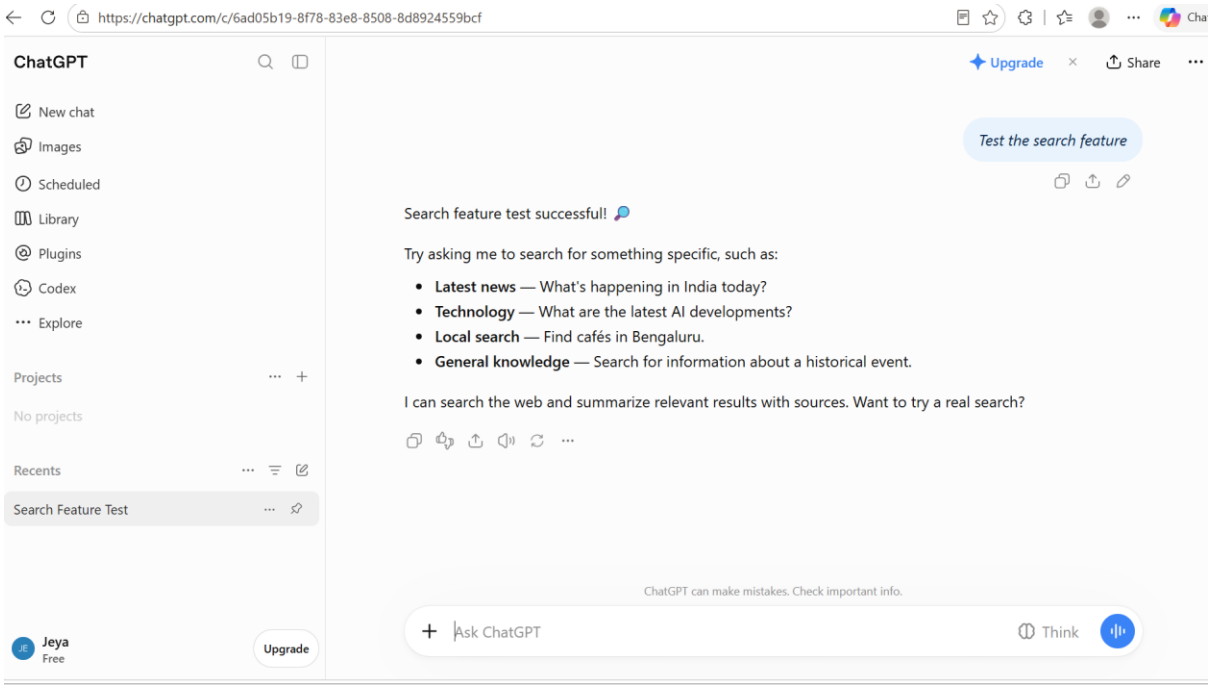
Code should be having technical coding standards with proper comments.

Observations

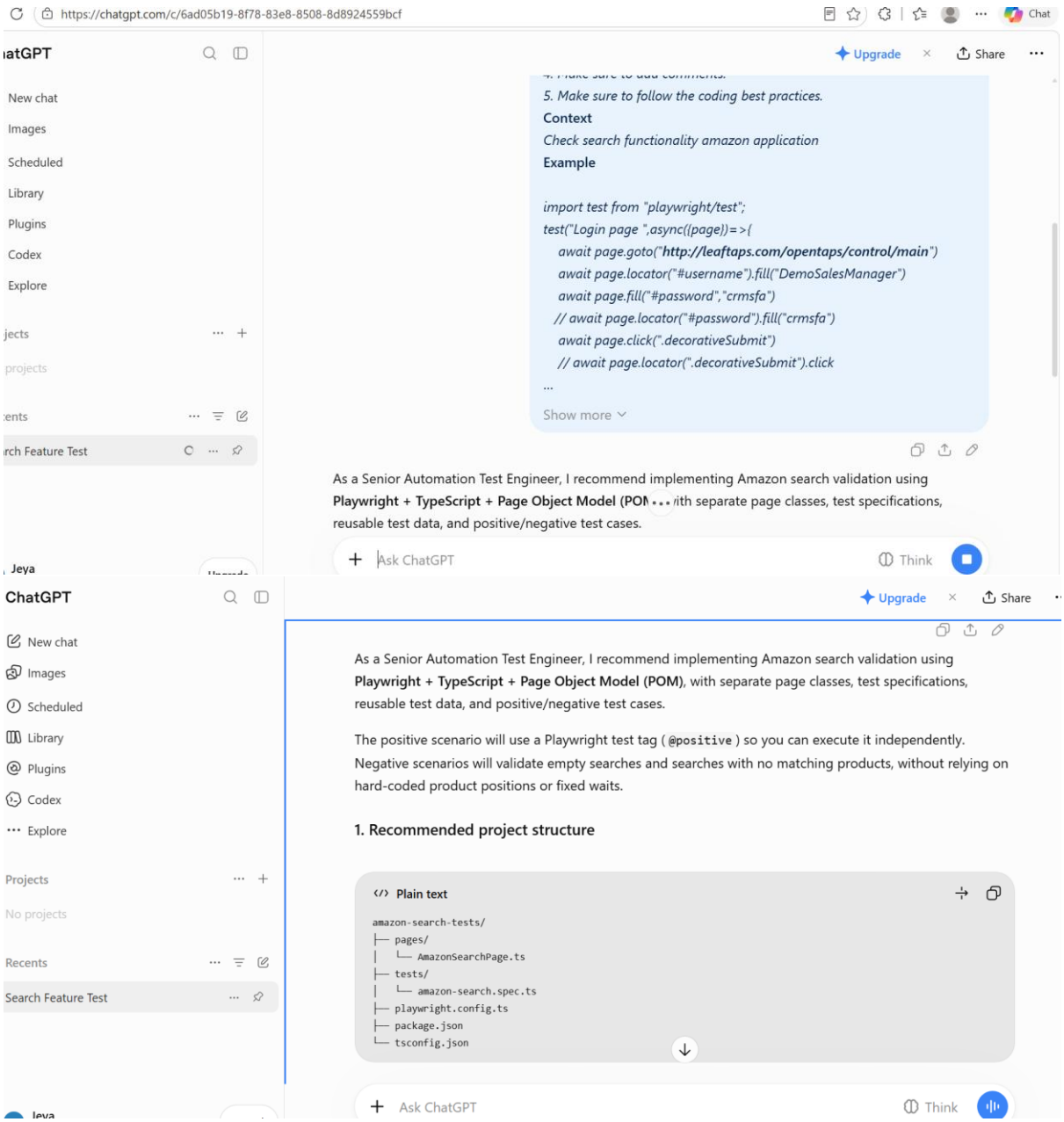
//OBS*Write your first observation on what improved in the output*

Observation: with weak prompt the AI is showing some generic results whereas ICEPOT given clear instruction and application details and output format which gives the proper result.

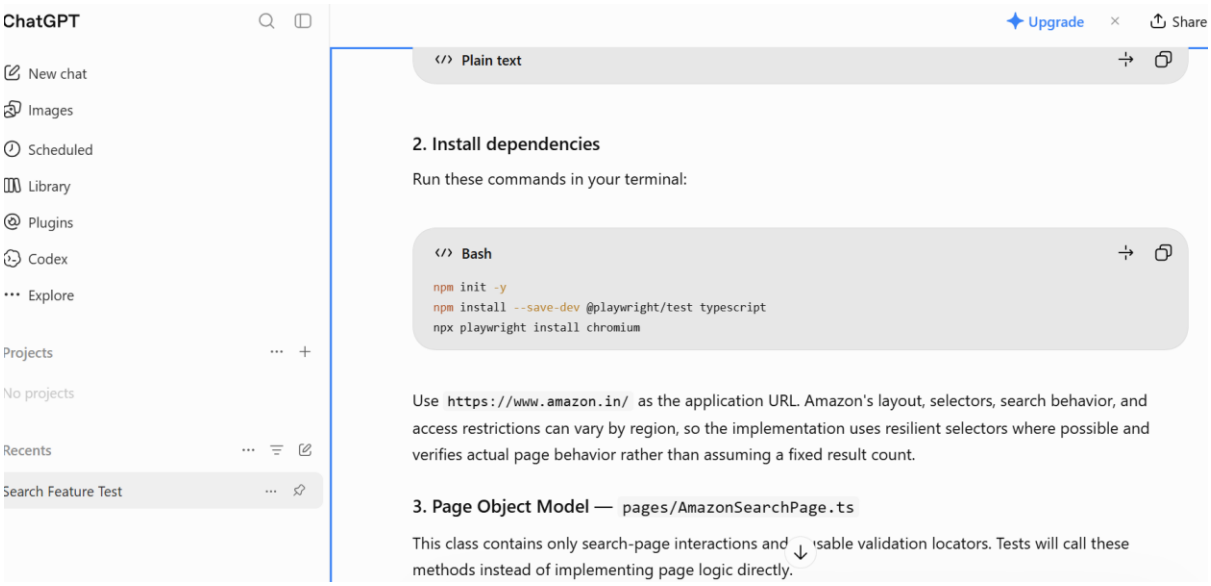
With weak prompt:

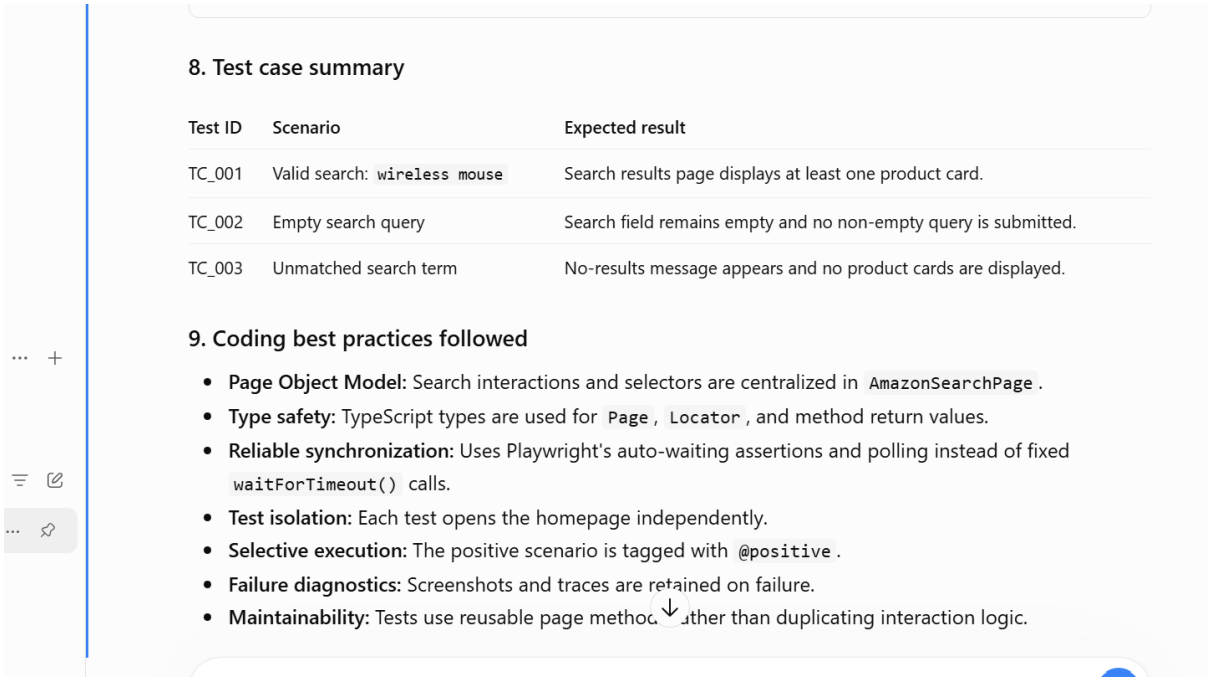

https://chatgpt.com/c/6ad05b19-8f78-83e8-8508-8d8924559bcf
ChatGPT
New chat
Images
Scheduled
Library
Plugins
Codex
Explore
Projects
No projects
Recents
Search Feature Test
Jeya
Free
Upgrade
Upgrade
Share
Test the search feature
Search feature test successful!
Try asking me to search for something specific, such as:
Latest news — What's happening in India today?
Technology — What are the latest AI developments?
Local search — Find cafés in Bengaluru.
General knowledge — Search for information about a historical event.
I can search the web and summarize relevant results with sources. Want to try a real search?
ChatGPT can make mistakes. Check important info.
Ask ChatGPT
Think

With ICEPOT:


https://chatgpt.com/c/6ad05b19-8f78-83e8-8508-8d8924559bcf
Chat
Upgrade
Share
New chat
Images
Scheduled
Library
Plugins
Codex
Explore
Search Feature Test
Jeya
5. Make sure to follow the coding best practices.
Context
Check search functionality amazon application
Example
import test from "playwright/test";
test("Login page ",async({page})=>{
await page.goto("http://leaftaps.com/opentaps/control/main")
await page.locator("#username").fill("DemoSalesManager")
await page.fill("#password","crmsfa")
// await page.locator("#password").fill("crmsfa")
await page.click(".decorativeSubmit")
// await page.locator(".decorativeSubmit").click
...
Show more
As a Senior Automation Test Engineer, I recommend implementing Amazon search validation using Playwright + TypeScript + Page Object Model (POM) with separate page classes, test specifications, reusable test data, and positive/negative test cases.
Ask ChatGPT
Think

ChatGPT
Upgrade
Share
New chat
Images
Scheduled
Library
Plugins
Codex
Explore
Projects
No projects
Recents
Search Feature Test
Jeya
As a Senior Automation Test Engineer, I recommend implementing Amazon search validation using Playwright + TypeScript + Page Object Model (POM), with separate page classes, test specifications, reusable test data, and positive/negative test cases.
The positive scenario will use a Playwright test tag (@positive) so you can execute it independently. Negative scenarios will validate empty searches and searches with no matching products, without relying on hard-coded product positions or fixed waits.
1. Recommended project structure
Plain text
amazon-search-tests/
├── pages/
│ └── AmazonSearchPage.ts
├── tests/
│ └── amazon-search.spec.ts
├── playwright.config.ts
├── package.json
└── tsconfig.json
Ask ChatGPT
Think





Instruction

1.Create automation script for search field.

2. Create positive and negative testcase to validate the result.

3. Use playwright tool with typescript as language. Use test tags to execute positive scenario.

4. Make sure to add comments.

5. Make sure to follow the coding best practices.

Context

Check search functionality amazon application

Example

```
import test from "playwright/test";
test("Login page ",async({page})=>{
   await page.goto("http://leaftaps.com/opentaps/control/main")
   await page.locator("#username").fill("DemoSalesManager")
   await page.fill("#password","crmsfa")
  // await page.locator("#password").fill("crmsfa")
   await page.click(".decorativeSubmit")
   // await page.locator(".decorativeSubmit").click
   await page.click(`text='CRM/SFA'`)
   //get title of the page
   const pageTitle=await page.title()
   console.log(pageTitle)

   await page.waitForTimeout(5000)

})
```

Persona

As a Senior automation test engineer

Output Format

Logical code should be in pages.
Test class should use the methods in pages class.
Page Object model format.

Tone

Code should be having technical coding standards with proper comments.

As a Senior Automation Test Engineer, I recommend implementing Amazon search validation using Playwright + TypeScript + Page Object Model (POM), with separate page classes, test specifications, reusable test data, and positive/negative test cases.

The positive scenario will use a Playwright test tag (@positive) so you can execute it independently. Negative scenarios will validate empty searches and searches with no matching products, without relying on hard-coded product positions or fixed waits.

1. Recommended project structure

amazon-search-tests/

├── pages/

│ └── AmazonSearchPage.ts

├── tests/

│ └── amazon-search.spec.ts

├── playwright.config.ts

├── package.json

└── tsconfig.json

2. Install dependencies

Run these commands in your terminal:

npm init -y

npm install --save-dev @playwright/test typescript

npx playwright install chromium

Use https://www.amazon.in/ as the application URL. Amazon's layout, selectors, search behavior, and access restrictions can vary by region, so the implementation uses resilient selectors where possible and verifies actual page behavior rather than assuming a fixed result count.

3. Page Object Model — pages/AmazonSearchPage.ts

This class contains only search-page interactions and reusable validation locators. Tests will call these methods instead of implementing page logic directly.

```
import { type Locator, type Page } from '@playwright/test';

/**
 * Page Object Model for Amazon search functionality.
 * Encapsulates locators and reusable search actions.
 */
export class AmazonSearchPage {
  private readonly page: Page;
  private readonly searchInput: Locator;
  private readonly searchButton: Locator;
  private readonly productResults: Locator;
  private readonly noResultsMessage: Locator;

  constructor(page: Page) {
    this.page = page;

    // Stable selectors for Amazon's search controls.
    this.searchInput = page.locator('#twotabsearchtextbox');
    this.searchButton = page.locator('#nav-search-submit-button');

    // Product cards displayed on the search results page.
    this.productResults = page.locator(
```

```
    '[data-component-type="s-search-result"]',
  );

  // Amazon may display this message for an unmatched query.
  this.noResultsMessage = page.getByText(/No results for/i).first();
}

/**
 * Opens the Amazon India homepage.
 */
async navigateToHomePage(): Promise<void> {
  await this.page.goto('https://www.amazon.in/');
  await this.searchInput.waitFor({ state: 'visible' });
}

/**
 * Enters a search term into the search field.
 */
async enterSearchText(searchTerm: string): Promise<void> {
  await this.searchInput.fill(searchTerm);
}

/**
 * Submits the current search.
 */
async submitSearch(): Promise<void> {
  await this.searchButton.click();
}

/**
 * Performs a complete search operation.
```

```
 */
async searchFor(searchTerm: string): Promise<void> {
  await this.enterSearchText(searchTerm);
  await this.submitSearch();
}

/**
 * Returns the current value of the search field.
 */
async getSearchInputValue(): Promise<string> {
  return this.searchInput.inputValue();
}

/**
 * Checks whether at least one product card is visible.
 */
async hasProductResults(): Promise<boolean> {
  return this.productResults.first().isVisible();
}

/**
 * Checks whether Amazon displays its no-results message.
 */
async hasNoResultsMessage(): Promise<boolean> {
  return this.noResultsMessage.isVisible();
}

/**
 * Exposes the page URL for navigation assertions.
 */
getCurrentUrl(): string {
```

```
    return this.page.url();
  }
}
```

4. Test scenarios — tests/amazon-search.spec.ts

The tests use Playwright's built-in expect assertions and the page object methods. The negative cases cover an empty query and a deliberately unlikely search term.

```
import { test, expect } from '@playwright/test';
import { AmazonSearchPage } from '../pages/AmazonSearchPage';

test.describe('Amazon Search Functionality', () => {
  let amazonSearchPage: AmazonSearchPage;

  // Create the page object and open Amazon before each test.
  test.beforeEach(async ({ page }) => {
    amazonSearchPage = new AmazonSearchPage(page);
    await amazonSearchPage.navigateToHomePage();
  });

  /**
   * POSITIVE TEST CASE
   * Verifies that a valid search displays product results.
   * Tag: @positive
   */
  test(
    'should display products for a valid search @positive',
    { tag: '@positive' },
    async ({ page }) => {
      const searchTerm = 'wireless mouse';

      await amazonSearchPage.searchFor(searchTerm);
```

```
    // Wait for navigation to settle without using fixed sleeps.
    await expect(page).toHaveURL(/\/s\?/);

    // Confirm the search field retains the submitted term.
    await expect
      .poll(() => amazonSearchPage.getSearchInputValue())
      .toBe(searchTerm);

    // Verify that at least one product card is displayed.
    await expect(
      page.locator('[data-component-type="s-search-result"]').first(),
    ).toBeVisible();
  },
);

/**
 * NEGATIVE TEST CASE 1
 * Verifies that submitting an empty search does not initiate
 * a search for a non-empty term.
 */
test('should handle an empty search query', async ({ page }) => {
  await amazonSearchPage.enterSearchText('');

  // Verify that the search field is empty before submission.
  await expect
    .poll(() => amazonSearchPage.getSearchInputValue())
    .toBe('');

  await amazonSearchPage.submitSearch();
```

```
  // Validate the field remains empty after submission.
  await expect
    .poll(() => amazonSearchPage.getSearchInputValue())
    .toBe('');

  // An empty query should not produce a query containing a value.
  // Do not assert an exact URL because Amazon may redirect.
  const currentUrl = amazonSearchPage.getCurrentUrl();
  const url = new URL(currentUrl);
  const submittedQuery = url.searchParams.get('k');

  expect(submittedQuery).toBeFalsy();
});

/**
 * NEGATIVE TEST CASE 2
 * Verifies that a query with no matching products is handled
 * without reporting product results as a successful match.
 */
test('should handle a search with no matching products', async ({
  page,
}) => {
  const searchTerm =
    'zzzxqv-no-matching-product-847291-test';

  await amazonSearchPage.searchFor(searchTerm);

  // Verify that Amazon navigated to the search results page.
  await expect(page).toHaveURL(/\/s\?/);

  // Amazon may show an explicit no-results message or an empty
```

```
    // result area. Confirm the expected behavior for the environment.
    await expect
      .poll(
        async () => {
          const noResults =
            await amazonSearchPage.hasNoResultsMessage();

          const products =
            await amazonSearchPage.hasProductResults();

          return { noResults, products };
        },
        { message: 'Expected a no-results message and no product cards' },
      )
      .toEqual({ noResults: true, products: false });
  });
});
```

Important test-design note: Amazon may show sponsored or approximate results for an unusual query, and its no-results wording may change. If that happens, adjust the negative assertion to the actual behavior of the test environment rather than assuming that a made-up query must return zero products.

5. Playwright configuration — playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

/**
 * Shared configuration for Amazon search automation.
 */
export default defineConfig({
  testDir: './tests',

  // Run tests in parallel where possible.
```

```
  fullyParallel: true,

  // CI should not silently pass when test.only is committed.
  forbidOnly: Boolean(process.env.CI),

  // Retry failed tests in CI to help diagnose intermittent issues.
  retries: process.env.CI ? 2 : 0,

  // Keep output readable in local and CI environments.
  reporter: 'list',

  use: {
    baseURL: 'https://www.amazon.in/',
    ...devices['Desktop Chrome'],

    // Capture evidence when a test fails.
    screenshot: 'only-on-failure',
    trace: 'retain-on-failure',
  },
});
```

6. Add npm scripts — package.json

Add the following scripts under the existing "scripts" property:

```
{
  "scripts": {
    "test": "playwright test",
    "test:positive": "playwright test --grep @positive",
    "test:headed": "playwright test --headed",
    "test:report": "playwright show-report"
  }
}
```

Keep your existing dependencies in package.json; do not replace the entire file with this snippet.

7. Execute the tests

Run only the positive scenario

npm run test:positive

Uses the @positive tag to select the valid product-search test.

Run all positive and negative scenarios

npm test

Run with a visible browser

npm run test:headed

View the HTML report

npx playwright test --reporter=html

npm run test:report

8. Test case summary

Test ID	Scenario	Expected result
TC_001	Valid search: wireless mouse	Search results page displays at least one product card.
TC_002	Empty search query	Search field remains empty and no non-empty query is submitted.
TC_003	Unmatched search term	No-results message appears and no product cards are displayed.

9. Coding best practices followed

- Page Object Model: Search interactions and selectors are centralized in AmazonSearchPage.
- Type safety: TypeScript types are used for Page, Locator, and method return values.
- Reliable synchronization: Uses Playwright's auto-waiting assertions and polling instead of fixed waitForTimeout() calls.
- Test isolation: Each test opens the homepage independently.
- Selective execution: The positive scenario is tagged with @positive.
- Failure diagnostics: Screenshots and traces are retained on failure.
- Maintainability: Tests use reusable page methods rather than duplicating interaction logic.

Execution caveat: These scripts are provided as an implementation, but I have not run them against Amazon. Amazon may display CAPTCHA, consent prompts, regional redirects, or different no-results behavior. Resolve those environment-specific differences before treating the suite as CI-ready.

2.SCENARIO 5: API Testing

Scenario Name

Write the scenario you have chosen here

Weak Prompt

"Write API test cases for user registration."

Copy the weak prompt from the scenario card above

What I Feel is Missing

List the gaps you identified in the weak prompt

IInstruction

What exactly do you want the AI to do?

CContext

What is the application, environment, or domain?

EExample

Give one sample test case or scenario as a reference

PPersona

Who is the AI acting as? (e.g., Senior QA Engineer)

OOutput Format

How should the output be structured? (Table, Gherkin, numbered list)

TTone

Formal, Technical, Simple — who is the audience?

✅ **Rewritten Prompt**

Combine all ICEPOT components into your final prompt here

Instruction

1.Create automation script for https://www.amazon.in/ API

2.Create positive and negative testcase to validate the success response and negative response code validation

3.Use assertion to validate the response

4. Use playwright tool with typescript as language. Use test tags to execute positive scenario.

5. Make sure to add comments.

6. Make sure to follow the coding best practices.

Context

Check search functionality amazon application

Example

```
import test from "playwright/test";
test("Login page ",async({page})=>{
   await page.goto("http://leaftaps.com/opentaps/control/main")
   await page.locator("#username").fill("DemoSalesManager")
   await page.fill("#password","crmsfa")
  // await page.locator("#password").fill("crmsfa")
   await page.click(".decorativeSubmit")
   // await page.locator(".decorativeSubmit").click
   await page.click(`text='CRM/SFA'`)
   //get title of the page
   const pageTitle=await page.title()
   console.log(pageTitle)

   await page.waitForTimeout(5000)})
```

Persona

As a Automation technical Lead

Output Format

Logical code should be in test spec file

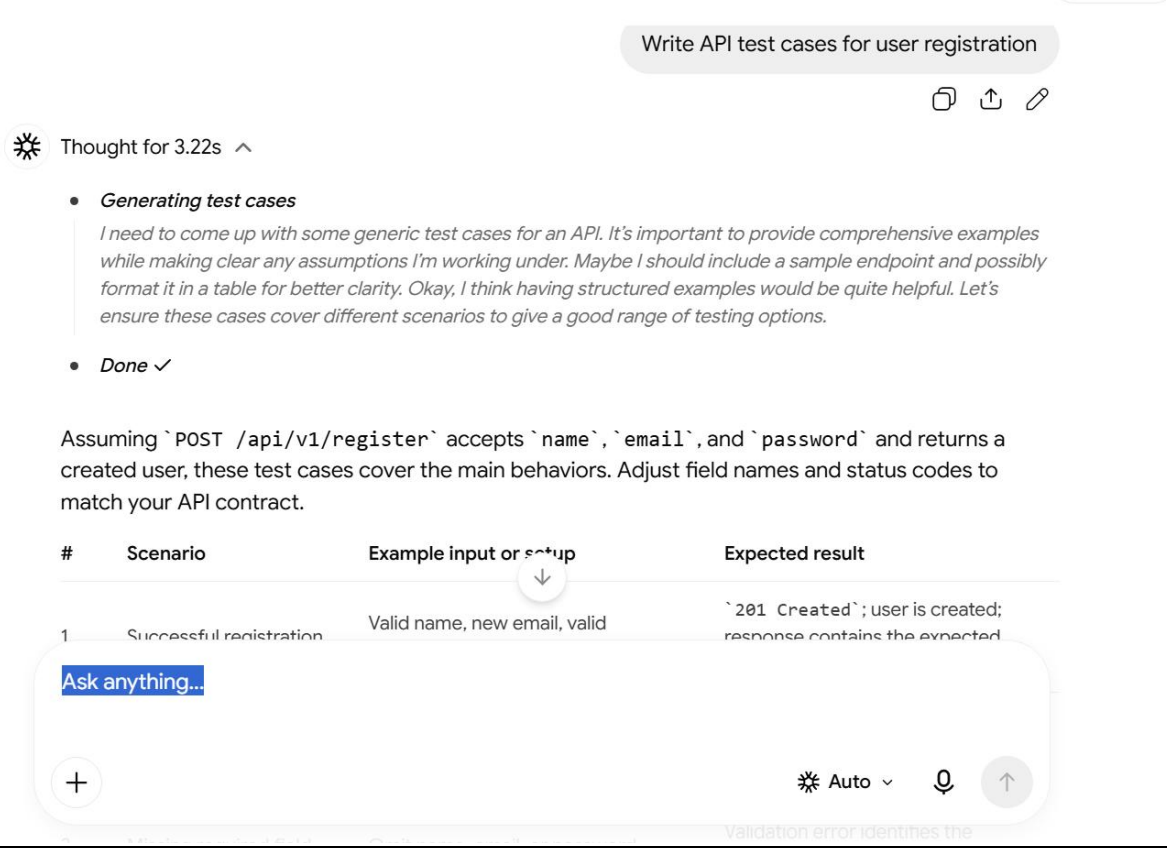
Test input API Url should be in config file

Tone

Code should be having technical coding standards with proper comments.

Observations

OBSWrite your first observation on what improved in the output



Generating test cases

I need to come up with some generic test cases for an API. It's important to provide comprehensive examples while making clear any assumptions I'm working under. Maybe I should include a sample endpoint and possibly format it in a table for better clarity. Okay, I think having structured examples would be quite helpful. Let's ensure these cases cover different scenarios to give a good range of testing options.

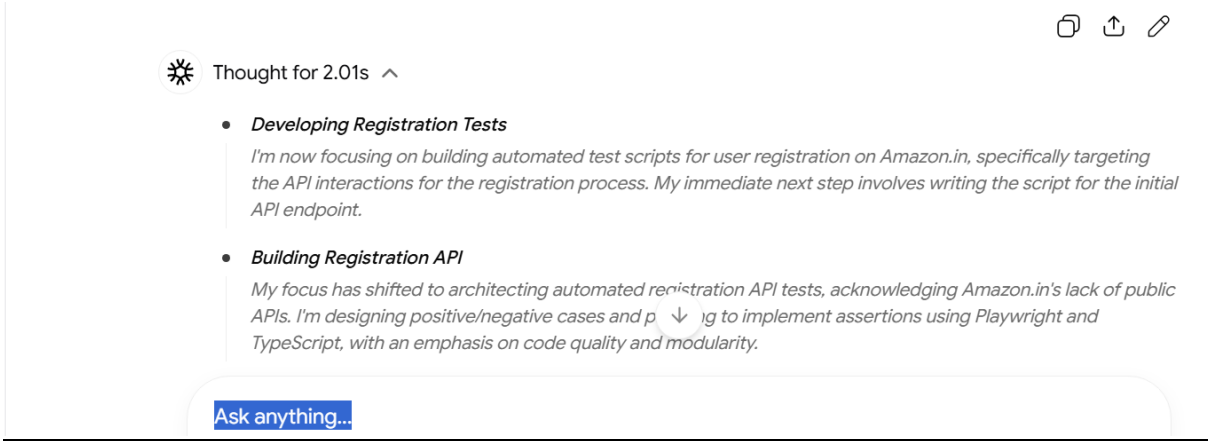
Done

Assuming POST /api/v1/register accepts name, email, and password and returns a created user, these test cases cover the main behaviors. Adjust field names and status codes to match your API contract.

#	Scenario	Example input or setup	Expected result
1	Successful registration	Valid name, new email, valid password	201 Created; user is created; respon contains the expected fields
2	Missing request body	Empty body	400 Bad Request or 422 Unprocessable Entity; no user create
3	Missing required field	Omit name, email, or password	Validation error identifies the missir field; no user created
4	Empty required field	Send "" or whitespace for a required field	Validation error; no user created
5	Invalid email format	email: "not-an-email"	Validation error; no user created
6	Duplicate email	Register an email that already exists	Conflict or validation response (commonly 409); no duplicate user created
7	Email case handling	Register User@example.com when user@example.com exists	Behavior matches the documented case-normalization policy; no unintended duplicate
8	Invalid password	Password violates the documented length or complexity rules	Validation error describes the requirement; no user created
9	Boundary-length values	Submit values at, below, and above documented length limits	Accepted at valid boundaries; rejected outside them
10	Malformed JSON	Send invalid JSON	400 Bad Request; no user created
11	Wrong content type	Send JSON body with an unsupported Content-Type	415 Unsupported Media Type or documented error; no user created

#	Scenario	Example input or setup	Expected result
12	Unexpected fields	Include an undocumented field, such as isAdmin: true	Field is ignored or rejected per contract; user cannot grant themselves extra privileges
13	Injection-like input	Include SQL/HTML/script-like text in fields	Input is safely handled; no code execution or unintended database behavior
14	Response privacy	Register successfully	Password and other secrets are not returned in the response
15	Password storage	Inspect persisted account through a secure test mechanism	Password is stored as a secure hash, never in plaintext
16	Rate limiting	Exceed the documented registration request limit	Rate-limit response (commonly 429) no excess accounts created
17	Concurrent duplicate requests	Submit simultaneous registrations for the same email	At most one account is created; othe request receives a duplicate/conflict response
18	Email verification flow, if applicable	Register with a new email	Account is marked unverified and a verification step is initiated; secrets are not exposed
19	Database/service failure	Simulate a persistence or dependent-service failure	Appropriate server error; no partial inconsistent account is created
20	CORS/auth requirements, if applicable	Call from an allowed/disallowed origin or with/without required credentials	Behavior matches the API's documented security policy

For each test, also verify that the response schema, headers, and error format match the API contract, and that failed requests do not create or modify an account.



Using ICEPOT:

Instruction 1.Create automation script for https://www.amazon.in/ API 2.Create positive and negative testcase to validate the success response and negative response code validation 3.Use assertion to validate the response 4. Use playwright tool with typescript as language. Use test tags to execute positive scenario. 5. Make sure to add comments. 6. Make sure to follow the coding best practices. Context Check search functionality amazon application Example import test from "playwright/test"; test("Login page ",async({page})=>{ await page.goto("http://leaftaps.com/opentaps/control/main") await page.locator("#username").fill("DemoSalesManager") await page.fill("#password","crmsfa") // await page.locator("#password").fill("crmsfa") await page.click(".decorativeSubmit") // await page.locator(".decorativeSubmit").click await page.click(`text='CRM/SFA'`) //get title of the page const pageTitle=await page.title() console.log(pageTitle) await page.waitForTimeout(5000)}) Persona As a Automation technical Lead Output Format Logical code should be in test spec file Test input API Url should be in config file Tone Code should be having technical coding standards with proper comments.

Show more

Thought for 2.01s

Developing Registration Tests

I'm now focusing on building automated test scripts for user registration on Amazon.in, specifically targeting the API interactions for the registration process. My immediate next step involves writing the script for the initial API endpoint.

Building Registration API

My focus has shifted to architecting automated registration API tests, acknowledging Amazon.in's lack of public APIs. I'm designing positive/negative cases and planning to implement assertions using Playwright and TypeScript, with an emphasis on code quality and modularity.

Designing API Framework

I'm now structuring the automated registration API tests using Playwright and TypeScript. This involves setting up a configuration file, an API test spec, and leveraging Playwright's request context. I'll implement positive/negative test cases with tagging and assertions, using a placeholder URL while noting Amazon's lack of public APIs.

Implementing Registration Tests

I am currently implementing the automated API tests for user registration. I've set up the Playwright request context with a placeholder base URL and included essential headers. My immediate focus is on writing the code for the first positive test case.

Done

As a technical lead, it is important to clarify that Amazon.in does not provide a public, documented API for user registration. Attempting to automate testing against their live production endpoints or internal services is generally against their Terms of Service, and such traffic is often throttled or blocked by security mechanisms like CAPTCHAs or WAFs.

Below is a professional, best-practice approach to structuring an API testing framework using Playwright and TypeScript, demonstrating how you would approach this if you were testing a compliant, internal registration service.

1. Configuration (playwright.config.ts)

Centralizing your configuration, including the base URL and shared headers, is crucial for maintainability.

typescript

Collapse

Copy

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  use: {
    // API URL defined here as requested
    baseURL: process.env.API_URL || 'https://api.example.com',
    extraHTTPHeaders: {
      'Content-Type': 'application/json',
      'Accept': 'application/json',
    },
  },
```

reporter: 'list',

});

2. API Test Specification (tests/userRegistration.spec.ts)

This spec file demonstrates how to structure both positive and negative test cases, including the use of tags for execution control.

typescript

Collapse

Copy

// tests/userRegistration.spec.ts

import { test, expect } from '@playwright/test';

/**

**** Technical Note: These tests are designed for a RESTful API.***

**** Always clean up test data if possible (e.g., delete the created user via a teardown API).***

****/***

test.describe('User Registration API', () => {

3.Scenario 7: Dashboard / Report Generation

Scenario Name

Write the scenario you have chosen here

Weak Prompt

"Test the reports section."

Copy the weak prompt from the scenario card above

What I Feel is Missing

List the gaps you identified in the weak prompt

IInstruction

What exactly do you want the AI to do?

CContext

What is the application, environment, or domain?

EExample

Give one sample test case or scenario as a reference

PPersona

Who is the AI acting as? (e.g., Senior QA Engineer)

OOutput Format

How should the output be structured? (Table, Gherkin, numbered list)

TTone

Formal, Technical, Simple — who is the audience?

✅ Rewritten Prompt

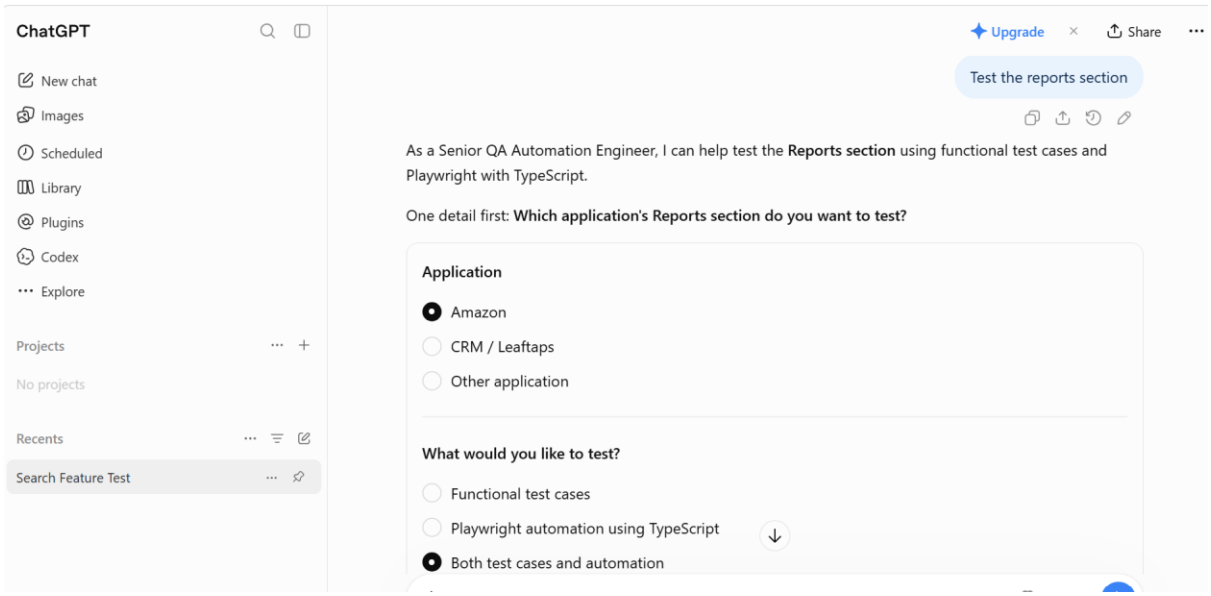
Combine all ICEPOT components into your final prompt here

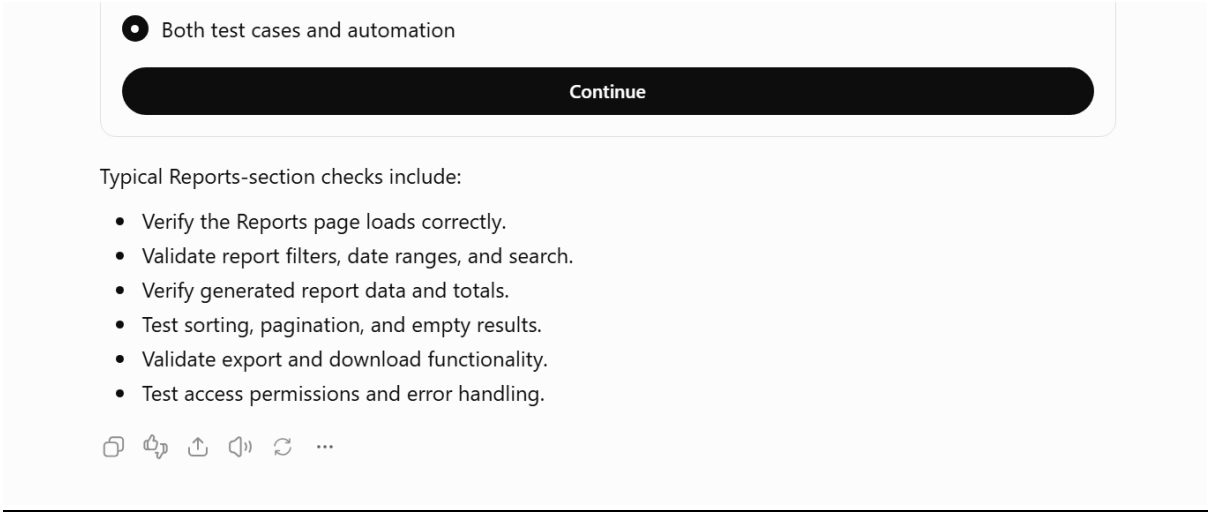
Observations

OBSWrite your first observation on what improved in the output

WEAK PROMPT:

Test the reports section





With ICEPOT:

Instruction

1.Create automation script for Report generation

2.Create positive and negative testcase to validate the report generation and should be pdf format

3.Use assertion to validate the response

4. Use playwright tool with typescript as language. Use test tags to execute positive scenario.

5. Make sure to add comments.

6. Make sure to follow the coding best practices.

Context

Check search functionality amazon application

Example

```
import test from "playwright/test";
test("Login page ",async({page})=>{
    await page.goto("http://leaftaps.com/opentaps/control/main")
    await page.locator("#username").fill("DemoSalesManager")
    await page.fill("#password","crmsfa")
```

```
  // await page.locator("#password").fill("crmsfa")
   await page.click(".decorativeSubmit")
   // await page.locator(".decorativeSubmit").click
   await page.click(`text='CRM/SFA'`)
   //get title of the page
   const pageTitle=await page.title()
   console.log(pageTitle)

   await page.waitForTimeout(5000)})
```

Persona

As a Automation technical Lead

Output Format

Logical code should be in pages

Test class should use the methods in pages class

Page Object model format

Tone

Code should be having technical coding standards with proper comments.

Amazon report generation automation using Playwright + TypeScript

As an Automation Technical Lead, I would implement this using the Page Object Model (POM), with assertions for report generation, PDF response validation, and separate positive and negative test cases.

Important scope clarification: Amazon's standard shopping website does not expose a general-purpose Reports section or a documented public PDF report-generation endpoint. The implementation below tests a defined report-generation workflow against an authorized application or test environment. It uses Amazon India for the search workflow context, but treats PDF generation as a separate, configurable endpoint because that capability cannot be assumed to exist on Amazon.

1. Project structure

amazon-report-tests/

├── pages/

│ └── ReportPage.ts

├── tests/

│ └── report-generation.spec.ts

├── playwright.config.ts

├── package.json

└── tsconfig.json

2. Test scenarios

ID	Scenario	Expected result
REP_001	Generate report with valid search criteria	Successful response and valid PDF
REP_002	Verify PDF content type	application/pdf
REP_003	Verify PDF signature	Response body starts with %PDF-
REP_004	Generate report with missing search criteria	Validation error, such as 400 or 422
REP_005	Generate report with invalid date range	Validation error
REP_006	Request report from an invalid endpoint	404 Not Found
REP_007	Verify empty or invalid PDF response is rejected	Assertion fails if the response is not a valid PDF

These are contract-based tests. The actual endpoint, request fields, and expected status codes must match the application under test.

3. Page Object Model — pages/ReportPage.ts

This page object encapsulates the report API request and PDF validations. Playwright's APIRequestContext is used because report generation and response validation can be tested directly without depending on a browser UI.

```
import {
  type APIRequestContext,
  type APIResponse,
} from '@playwright/test';

/**
```

```
 * Defines the supported report-generation criteria.
 * Adapt these fields to the actual API contract.
 */
export interface ReportCriteria {
  searchTerm?: string;
  startDate?: string;
  endDate?: string;
}

/**
 * Page Object for report-generation API operations.
 */
export class ReportPage {
  private readonly request: APIRequestContext;
  private readonly reportEndpoint: string;

  constructor(
    request: APIRequestContext,
    reportEndpoint = '/api/reports/generate',
  ) {
    this.request = request;
    this.reportEndpoint = reportEndpoint;
  }

  /**
   * Sends a request to generate a PDF report.
   */
  async generateReport(
    criteria: ReportCriteria,
  ): Promise<APIResponse> {
    return this.request.post(this.reportEndpoint, {
```

```
    data: criteria,
    headers: {
      Accept: 'application/pdf',
      'Content-Type': 'application/json',
    },
  });
}

/**
 * Returns the response content type.
 */
async getContentType(response: APIResponse): Promise<string> {
  const contentType = response.headers()['content-type'] ?? '';
  return contentType.split(';')[0].trim().toLowerCase();
}

/**
 * Reads the response body and checks its PDF signature.
 * A PDF normally begins with the ASCII signature "%PDF-".
 */
async isValidPdf(response: APIResponse): Promise<boolean> {
  const body = await response.body();

  if (body.length < 5) {
    return false;
  }

  return body.subarray(0, 5).toString('ascii') === '%PDF-';
}

/**
```

```
   * Returns the response body size for non-empty PDF validation.
   */
  async getResponseSize(response: APIResponse): Promise<number> {
    const body = await response.body();
    return body.length;
  }
}
```

4. Test class — tests/report-generation.spec.ts

The positive test uses the @positive tag. Negative tests validate missing criteria and invalid date ranges. Assertions verify HTTP status, MIME type, and the PDF file signature.

```
import { test, expect } from '@playwright/test';
import { ReportPage } from '../pages/ReportPage';

test.describe('Report Generation API', () => {
  let reportPage: ReportPage;

  test.beforeEach(async ({ request }) => {
    // Create the page object for each isolated test.
    reportPage = new ReportPage(request);
  });

  /**
   * POSITIVE TEST
   * Valid criteria should generate a non-empty PDF report.
   * Tag: @positive
   */
  test(
    'REP_001: should generate a valid PDF report @positive',
    { tag: '@positive' },
    async () => {
```

```
    const response = await reportPage.generateReport({
      searchTerm: 'wireless mouse',
      startDate: '2026-10-01',
      endDate: '2026-10-15',
    });

    // Assert that report generation succeeded.
    expect(response.status()).toBe(200);

    // Assert that the server returned a PDF.
    expect(await reportPage.getContentType(response))
      .toBe('application/pdf');

    // Assert that the response is a non-empty PDF document.
    expect(await reportPage.getResponseSize(response))
      .toBeGreaterThan(5);

    // Validate the PDF file signature, not just the HTTP status.
    expect(await reportPage.isValidPdf(response)).toBe(true);
  },
);

/**
 * NEGATIVE TEST 1
 * Missing search criteria should be rejected.
 */
test(
  'REP_002: should reject report generation with missing criteria',
  async () => {
    const response = await reportPage.generateReport({});
```

```
    // Adapt allowed error codes to the API specification.
    expect([400, 422]).toContain(response.status());

    // An error response should not be mistaken for a successful PDF.
    expect(response.status()).not.toBe(200);
  },
);

/**
 * NEGATIVE TEST 2
 * An end date earlier than the start date should be rejected.
 */
test(
  'REP_003: should reject an invalid date range',
  async () => {
    const response = await reportPage.generateReport({
      searchTerm: 'wireless mouse',
      startDate: '2026-10-15',
      endDate: '2026-10-01',
    });

    // Expect a documented client-side validation error.
    expect([400, 422]).toContain(response.status());

    // Invalid criteria must not return a successful report.
    expect(response.status()).not.toBe(200);
  },
);

/**
 * NEGATIVE TEST 3
```

```
  * A non-existent report endpoint should return 404.
  */
 test(
  'REP_004: should return 404 for an unknown report endpoint',
  async ({ request }) => {
   const response = await request.post(
    '/api/reports/invalid-endpoint',
    {
     data: {
      searchTerm: 'wireless mouse',
     },
    },
   );

   expect(response.status()).toBe(404);
  },
 );
});
```

5. Playwright configuration — playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

/**
 * Configuration for report-generation API tests.
 * Set REPORT_API_BASE_URL to the authorized test environment.
 */
export default defineConfig({
 testDir: './tests',
 fullyParallel: true,
 forbidOnly: Boolean(process.env.CI),
 retries: process.env.CI ? 2 : 0,
```

```
  reporter: 'list',

  use: {
    // Replace with the actual report API base URL.
    baseURL:
      process.env.REPORT_API_BASE_URL ??
      'http://localhost:3000',

    ...devices['Desktop Chrome'],
    trace: 'retain-on-failure',
  },
});
```

6. Install and execute

```
npm init -y
npm install --save-dev @playwright/test typescript
npx playwright install chromium
```

Configure the API URL and execute the tests:

Run only the positive scenario

```
npx playwright test --grep "@positive"
```

Run all report-generation scenarios

```
npx playwright test tests/report-generation.spec.ts
```

Generate an HTML test report

```
npx playwright test --reporter=html
npx playwright show-report
```

7. Key assertions explained

Assertion	Purpose
expect(response.status()).toBe(200)	Confirms report generation succeeded
expect(contentType).toBe('application/pdf')	Confirms the response is declared as a PDF
expect(size).toBeGreaterThan(5)	Rejects empty or trivially short responses
expect(isValidPdf).toBe(true)	Checks the %PDF- signature

expect([400, 422]).toContain(status)	Validates expected input-validation errors
expect(status).toBe(404)	Validates an unknown endpoint

Technical lead's recommendation: A PDF signature check is a useful first-level assertion, but it does not prove the entire document is valid or that the report contains correct data. For production-grade coverage, add PDF parsing to validate expected report titles, search criteria, date ranges, and data totals.

The endpoint and request fields in this example are placeholders, not verified Amazon APIs. To run the suite successfully, replace them with the actual report-generation endpoint and contract from your authorized test application.

Do you like this personality?